Factors Influencing Student Performance with SQL

1. Problem Statement

1.1 Background

Educational institutions collect large volumes of student data — including attendance records, study habits, socio-economic backgrounds, and academic results. However, this data often remains underutilized because it is stored in scattered formats without a structured system to analyze it effectively.

1.2 Challenges Faced by Analysts

Educational analysts currently face the following key challenges:

- Data Fragmentation: Student performance data is spread across multiple files (Excel sheets, CSV exports, paper records), making it difficult to run unified analysis.
- Manual Processing: Analysts spend excessive time manually cleaning and merging datasets instead of focusing on extracting insights.
- Lack of Automated Reporting: There is no automated mechanism to flag at-risk students based on attendance or performance trends — reports are generated reactively, not proactively.
- Limited Query Capability: Without a structured database, complex analytical queries (e.g., correlation between study time and grades) are difficult or impossible to perform consistently.
- Inconsistent Data Quality: Missing values, duplicate records, and unstandardized formats lead to unreliable analysis results.
- No Centralized Access Control: Multiple stakeholders (administrators, analysts) access the same data without role-based permissions, creating security and integrity risks.

1.3 Problem Summary

> *There is no structured, automated SQL-based system that enables educational analysts to efficiently import, clean, query, and report on the factors influencing student academic performance — leading to delayed insights, poor data quality, and missed opportunities for early intervention.*

2. System Objectives

The following technical objectives define what the Student Performance Analysis System must achieve:

2.1 Primary Technical Objectives

Objective 1 — Centralized Data Storage

- Build a structured relational database (PostgreSQL/MySQL) to store all student performance records in a single, accessible location.
- Define a normalized schema that eliminates data redundancy and supports efficient querying.

Objective 2 — Automated Data Import & Cleaning Pipeline

- Develop scripts to automatically import datasets from CSV/Excel formats into the database.
- Implement data cleaning procedures to handle null values, remove duplicates, and standardize column formats.

Objective 3 — Accurate Analytical Query Engine

- Design and implement SQL queries that identify correlations between key factors (attendance, study time, socio-economic status) and student grades.
- Ensure query execution time remains under 2 seconds for datasets up to 100,000 records.
- Support filtering, grouping, and aggregation to enable flexible multi-factor analysis.

Objective 4 — Automated Reporting System

- Implement scheduled stored procedures that automatically generate nightly attendance risk reports.
- Enable proactive identification of at-risk students without manual intervention.
- Store generated reports for historical review and trend analysis.

Objective 5 — Role-Based Access & Security

- Implement SQL user roles to restrict data access based on user type (Administrator vs. Analyst).
- Ensure sensitive student data is protected through permission controls and encrypted connections.

Objective 6 — Exportable Results & Dashboards

- Allow analysts to export query results in CSV format for further use.
- Provide clear, tabular output for all analytical queries with summary statistics (mean, median, standard deviation).

2.2 Objectives Summary Table

#	Objective	Technical Goal	Success Metric
1	Centralized Storage	Normalized SQL schema	Zero data redundancy
2	Data Import & Cleaning	Automated ETL pipeline	< 5 min for 50MB file
3	Analytical Query Engine	Multi-factor SQL queries	< 2 sec per query
4	Automated Reporting	Scheduled stored procedures	Nightly report generated
5	Access Control	Role-based SQL permissions	2 roles defined & enforced
6	Exportable Results	CSV export + summary stats	Analyst self-service

3. System Architecture Design

The system follows a layered architecture pattern, where data flows through distinct processing stages from raw input files to final analytical outputs.

3.1 Architecture Overview

Layer 1 — Data Source

CSV Files | Excel Files | External Datasets (Kaggle)
Raw student performance data — unstructured, potentially dirty

▼ Data Ingestion Scripts (Python + pandas)

Layer 2 — Data Processing & Cleaning

Remove Duplicates | Handle Null Values | Standardize Formats
Tools: Python (pandas, SQLAlchemy)

▼ Load into Database

Layer 3 — Database Layer

PostgreSQL / MySQL — Relational Database
Tables: students | attendance | study_habits | socioeconomic | performance

▼ SQL Queries + Stored Procedures

Layer 4 — Analytics & Reporting Layer

Analytical Queries | Stored Procedures | Automated Reports
Scheduled nightly jobs | CSV export | Summary statistics

▼ Results Delivered To

Layer 5 — Presentation Layer (Users)

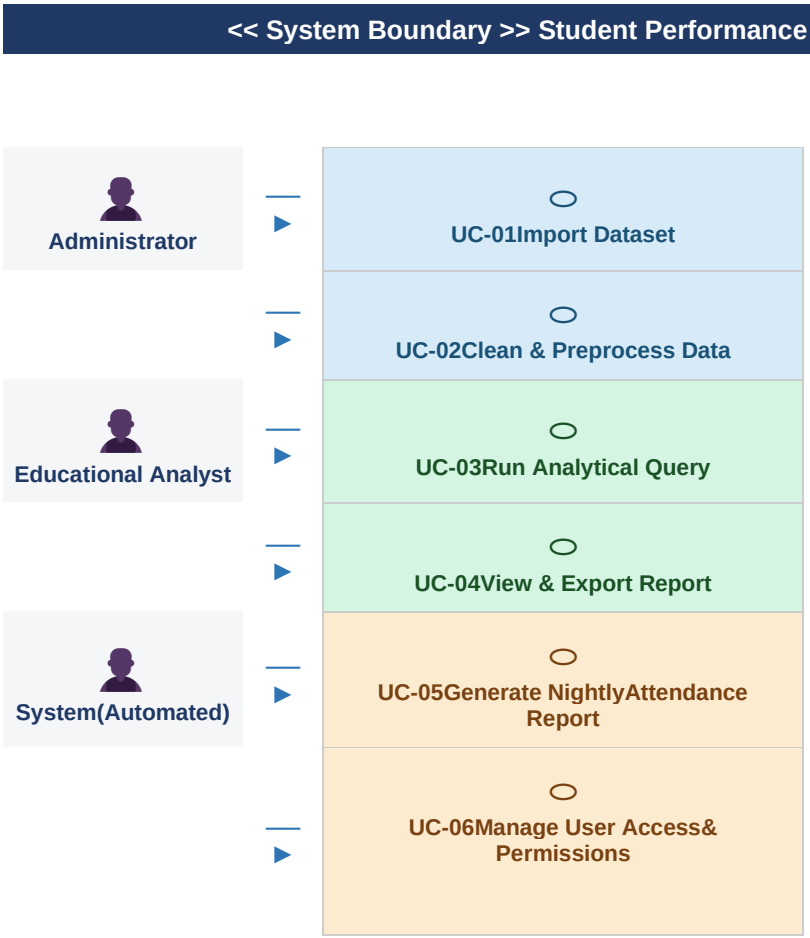
Administrator | Educational Analyst
View reports | Run queries | Export CSV | Monitor system health

3.2 Technology Stack

Component	Technology	Purpose
Database	PostgreSQL / MySQL	Store & query student data
Data Processing	Python (pandas, SQLAlchemy)	ETL pipeline & data cleaning
Automation	SQL Stored Procedures	Nightly scheduled reports
Version Control	GitHub	Code & documentation tracking
Export Format	CSV	Analyst result exports

4. Use Case Diagram

The following diagram illustrates how each actor interacts with the Student Performance Analysis System. Actors are shown on the left; use cases are inside the system boundary.



> *Note: Arrows (──►) represent associations between actors and use cases. The system boundary box encloses all internal system functions.*

4.1 Actor Descriptions

Actor	Role	Interacts With
Administrator	Manages data imports, user access, and system health	UC-01, UC-02, UC-06
Educational Analyst	Runs analytical queries and views/exports reports	UC-03, UC-04
System (Automated)	Executes scheduled nightly jobs without user input	UC-05

4.2 Use Case Summary

ID	Name	Actor	Summary
UC-01	Import Dataset	Administrator	Upload and store CSV/Excel files
UC-02	Clean & Preprocess Data	Administrator/System	Remove duplicates, handle nulls, standardize
UC-03	Run Analytical Query	Educational Analyst	Execute predefined SQL performance queries
UC-04	View & Export Report	Educational Analyst	View results and export to CSV format
UC-05	Generate Nightly Attendance Report	System (Automated)	Auto-run nightly job to flag at-risk students
UC-06	Manage User Access & Permissions	Administrator	Set role-based permissions for system users